Status	Finished
Started	Monday, 31 August 2026, 9:00 AM
Completed	Monday, 31 August 2026, 9:10 AM
Duration	9 mins 33 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Given a positive integer a, find the smallest positive integer b whose multiplication of each digit equals to a.

If there is no answer or the answer is not fit in 32-bit signed integer, then return 0.

Example 1

Input:

48

Output:

68

Example 2

Input:

15

Output:

35

For example:

Input	Result
48	68
15	35

Answer: (penalty regime: 0 %)

```
def smallest_factor_number(a):
    # Base cases for single digits
    if a < 10:
        return a

    digits = []

    # Try factorization using single digits from 9 down to 2
    for i in range(9, 1, -1):
        while a % i == 0:
            digits.append(str(i))
            a //= i

    # If a is not fully reduced to 1, it means a has a prime factor > 7 (e.g. 11, 13)
    if a > 1:
        return 0

    # Reverse the digits to get the smallest number (e.g., [8, 6] becomes "68")
    result_str = "".join(reversed(digits))
    result = int(result_str)

    # Check if the result fits in a 32-bit signed integer (up to 2,147,483,647)
```

```
    if result > 2**31 - 1:
        return 0

    return result

# Read input from the user
a = int(input())
print(smallest_factor_number(a))

```

	Input	Expected	Got	
✓	48	68	68	✓
✓	15	35	35	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

You are choreographing a circus show with various animals. For one act, you are given two kangaroos on a number line ready to jump in the positive direction.
•The first kangaroo starts at position x1 and moves at a speed v1 meters per jump.
•The second kangaroo starts at position x2 and moves at a speed of v2 meters per jump and x2 > x1
•You have to figure out to get both kangaroos at the same position at the same time as part of the show before k jumps. If it is possible, return YES, otherwise return NO.

Input Format:
x1-position of kangaroo1
v1-Speed of kangaroo1
x2-position of kangaroo2
v2-Speed of kangaroo2
k-jumps

Output Format:
Both kangaroos are at the same position within k jumps, YES, otherwise NO.

For example:

Input	Result
0 3 4 2 6	YES

Answer: (penalty regime: 0 %)

```
# Read inputs from the user
x1 = int(input())
v1 = int(input())
x2 = int(input())
v2 = int(input())
k = int(input())

# To catch up, kangaroo 1 must be strictly faster than kangaroo 2
if v1 <= v2:
    print("NO")
else:
    # Calculate the relative distance and relative speed
    distance_diff = x2 - x1
    speed_diff = v1 - v2

    # Check if they meet perfectly (remainder 0) and within the limit of k jumps
    if (distance_diff % speed_diff == 0) and (distance_diff // speed_diff < k):
        print("YES")
    else:
        print("NO")

```

	Input	Expected	Got	
✓	0 3 4 2 6	YES	YES	✓
✓	0 3 2 4 8	NO	NO	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Given a number N, find the next perfect square greater than N.

Input Format:

Integer input from stdin.

Output Format:

Perfect square greater than N.

Example Input:

10

Output:

16

For example:

Input	Result
10	16
20	25

Answer: (penalty regime: 0 %)

```
import math

# Read the integer input
N = int(input())

# Handle negative numbers or zero
if N < 0:
    print(0)
else:
    # Find the square root of N, round down, and add 1
    next_root = int(math.floor(N ** 0.5)) + 1

    # Calculate and print the next perfect square
    next_square = next_root * next_root
    print(next_square)

```

	Input	Expected	Got	
✓	10	16	16	✓
✓	20	25	25	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Write a program to find the count of the number of prime numbers in a specified range.

The starting and ending number of the range will be provided as input to the program.

Assumption: 2 <=starting number of the range<= ending number of the range<=7919

Example1: If the starting and ending number or the range is given as 2 and 20, the program must return 8, because there are 8 prime numbers in the specified range from 2 to 20. namely (2. 3. 5, 7, 11, 13, 17, 19)

Example2: If the starting and ending number of the range is given as 700 and 725, the program must return 3, because there are 3 prime numbers in the specified range from 700 to 725, namely (701, 709, 719)

For example:

Input	Result
2 20	8
700 725	3

Answer: (penalty regime: 0 %)

```
# Read the starting and ending values of the range
start = int(input())
end = int(input())

prime_count = 0

# Iterate through every number in the range (inclusive)
for num in range(start, end + 1):
    if num > 1:
        is_prime = True
        # Check for factors from 2 up to the square root of the number
        for i in range(2, int(num**0.5) + 1):
            if num % i == 0:
                is_prime = False
                break
        # If no factors were found, it's a prime number
        if is_prime:
            prime_count += 1

# Print the final count of prime numbers
print(prime_count)

```

	Input	Expected	Got	
✓	2 20	8	8	✓
✓	700 725	3	3	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Write a program to find the count of non-repeated digits in a given number N. The number will be passed to the program as an input of type int.

Assumption: The input number will be a positive integer number >= 1 and <= 25000.

Some examples are as below.

If the given number is 292, the program should return 1 because there is only 1 non-repeated digit '9' in this number

If the given number is 1015, the program should return 2 because there are 2 non-repeated digits in this number, '0', and '5'.

If the given number is 108, the program should return 3 because there are 3 non-repeated digits in this number, '1', '0', and '8'.

If the given number is 22, the function should return 0 because there are NO non-repeated digits in this number.

For example:

Input	Result
292	1
1015	2
108	3
22	0

Answer: (penalty regime: 0 %)

```
# Read the positive integer from standard input
N = int(input())

# Convert the integer to a string to easily inspect individual digits
num_str = str(N)

non_repeated_count = 0

# Check each unique digit present in the number string
for digit in set(num_str):
    # Count how many times this specific digit appears in the string
    if num_str.count(digit) == 1:
        non_repeated_count += 1

# Print the final count
print(non_repeated_count)

```

	Input	Expected	Got	
✓	292	1	1	✓
✓	1015	2	2	✓
✓	108	3	3	✓
✓	22	0	0	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Write a program that given an integer ‘n’, prints the number of integers that are less than or equal to ‘n’ and co-prime to ‘n’

Two integers a and b are said to be relatively prime or co-prime if the only positive integer that evenly divides both of them is 1. That is, the only common positive factor of the two numbers is 1. This is equivalent to their greatest common divisor being 1.

Input Format:

One line containing the value of 'n', where 1<=n<=10,000

Output Format:

One line containing the number of integers that are co-prime to n and less than or equal to 'n'

Sample Test Cases

Test Case 1

Input

10

Output

4

Test Case 2

Input

23

Output

22

Test Case 3

Input

11

Output

10

Answer: (penalty regime: 0 %)

```
import math

# Read the integer 'n' from standard input
n = int(input())

coprime_count = 0

# Check every integer from 1 up to n
for i in range(1, n + 1):
    # If the greatest common divisor is 1, they are co-prime
    if math.gcd(i, n) == 1:
        coprime_count += 1

# Print the final count
print(coprime_count)

```

	Input	Expected	Got	
✓	10	4	4	✓
✓	23	22	22	✓
✓	11	10	10	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

A Number is said to be Disarium number when the sum of its digit raised to the power of their respective positions becomes equal to the number itself. Write a program to print number is Disarium or not.

Input Format:

Single Integer Input from stdin.

Output Format:

Yes or No.

Example Input:

175

Output:

Yes

Explanation

1^1 + 7^2 +5^3 = 175

Example Input:

123

Output:

No

For example:

Input	Result
175	Yes
123	No

Answer: (penalty regime: 0 %)

```
# Read the number as a string to easily get positions and digits
num_str = input().strip()
num = int(num_str)

digit_sum = 0

# Loop through each digit with its 1-based position index
for index, digit_char in enumerate(num_str):
    digit = int(digit_char)
    position = index + 1

    # Raise the digit to the power of its position and add to the sum
    digit_sum += digit ** position

# Check if the calculated sum matches the original number
if digit_sum == num:
    print("Yes")
else:
    print("No")

```

	Input	Expected	Got	
✓	175	Yes	Yes	✓
✓	123	No	No	✓
✓	89	Yes	Yes	✓
✓	90	No	No	✓

	Input	Expected	Got	
✓	518	Yes	Yes	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Let's print a chessboard!

Write a program that takes input:

Integer N(represents the rows and columns of a chessboard) and also the starting character of the chessboard

Output Format

Print the chessboard as per the given examples

Sample Input / Output

Input:

2

W

Output:

WB

BW

Answer: (penalty regime: 0 %)

```
# Read the dimensions and the starting character from the input
N = int(input())
start_char = input().strip().upper()

# Set up the two alternative characters
char1 = start_char
char2 = 'B' if char1 == 'W' else 'W'

# Generate the chessboard row by row
for row in range(N):
    row_chars = []
    for col in range(N):
        # If the sum of row and col index is even, use char1; if odd, use char2
        if (row + col) % 2 == 0:
            row_chars.append(char1)
        else:
            row_chars.append(char2)

    # Print the row characters joined together without spaces
    print("".join(row_chars))

```

	Input	Expected	Got	
✓	2 W	WB BW	WB BW	✓
✓	3 B	BWB WBW BWB	BWB WBW BWB	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Write python program to print the following pattern based on input size.

Input:

3

Output:

```
  1
 2 3
4 5 6
```

For example:

Input	Result
4	1 2 3 4 5 6 7 8 9 10

Answer: (penalty regime: 0 %)

```
# Read the number of rows from the user
N = int(input())

# Initialize the starting number to print
current_number = 1

# Loop through each row from 1 up to N
for row in range(1, N + 1):
    # Calculate and print the leading spaces to align the pyramid
    print(" " * (N - row), end="")

    # Generate the numbers for the current row
    row_numbers = []
    for col in range(row):
        row_numbers.append(str(current_number))
        current_number += 1

    # Join the row numbers with a single space and print
    print(" ".join(row_numbers))

```

	Input	Expected	Got	
✓	3	1 2 3 4 5 6	1 2 3 4 5 6	✓
✓	4	1 2 3 4 5 6 7 8 9 10	1 2 3 4 5 6 7 8 9 10	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

An automorphic number is a number whose square ends with the number itself.

For example, 5 is an automorphic number because 5*5 =25. The last digit is 5 which same as

the given number.

If it is an automorphic number display “Automorphic” else display “Not Automorphic”.

Input Format:

Take a Integer from Keyboard

Output Format:

Print Automorphic if given number is Automorphic number, otherwise Not Automorphic

Example input:

5

Output:

Automorphic

Example input:

25

Output:

Automorphic

Example input:

7

Output:

Not Automorphic

Answer: (penalty regime: 0 %)

```
# Read the integer input from the user
num_str = input().strip()
num = int(num_str)

# Calculate the square of the number
square = num * num

# Check if the square ends with the original number string
if str(square).endswith(num_str):
    print("Automorphic")
else:
    print("Not Automorphic")

```

	Input	Expected	Got	
✓	5	Automorphic	Automorphic	✓
✓	625	Automorphic	Automorphic	✓
✓	7	Not Automorphic	Not Automorphic	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.